Integration Manual

for S32 SERDES Driver

Document Number: IM11SERDESASR4.4 Rev0000R4.0.2 Rev. 1.0

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

This integration manual describes the integration requirements for Serdes Driver for S32 microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSW	Basic Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
ECU	Electronic Control Unit
ISR	Interrupt Service Routine
N/A	Not Applicable
VLE	Variable Length Encoding

2.5 Reference List

#	Title	Version
1	Reference Manual	S32G2 Reference Manual, Rev. 7, February 2023
		S32G3 Reference Manual, Rev.3, Feb 2023
2	Datasheet	S32G2 Data Sheet, Rev 6, Dec 2022
		S32G3 Data Sheet, Rev 2, RC, Feb 2023
		VR5510 Data Sheet, Rev 5, April 2022
3	Errata	S32G2: Mask Set Errata for Mask 0P77B v3.0
		S32G2: Mask Set Errata for Mask 1P77B v1.0
		S32G3: Mask Set Errata for Mask 1P72B v2.1

Chapter 3

Building the driver

- Build Options
- Files required for compilation
- Setting up the plugins

This section describes the source files and various compilers, linker options used for building the driver.

It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

- GCC Compiler/Assembler/Linker Options
- GHS Compiler/Assembler/Linker Options
- DIAB Compiler/Assembler/Linker Options

The RTD driver files are compiled using:

- NXP GCC 9.2.0 20190812 (Build 1649 Revision gaf57174)
- Green Hills Multi 7.1.6d / Compiler 2020.1.4
- Wind River Diab Compiler 7.0.3

The compiler, assembler, and linker flags used for building the driver are explained below.

The TS_T40D11M40I2R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D11 identifies S32 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

3.1.1 GCC Compiler/Assembler/Linker Options

3.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioend -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned
-fomit-frame-pointer	Omit the frame pointer in functions that don't need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.

Compiler Option	Description
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	This option is only used to build test for generation Ram/↩ Stack size report. Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	This option is only used to build test for generation Ram/↩ Stack size report. Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DGCC	Predefine GCC as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.1.2 GCC Assembler Options

Assembler Option	Description
-X assembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

3.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
–entry=Reset_Handler	Specifies that the program entry point is Reset_Handler
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library

3.1.2 GHS Compiler/Assembler/Linker Options

3.1.2.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-thumb	Selects generating code that executes in Thumb state
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-C99	Use (strict ISO) C99 standard (without extensions)
–ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)
-Osize	Optimize for size
–gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory

Compiler Option	Description
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
–unsigned_chars	Let the type char be unsigned, like unsigned char
–unsigned_fields	Bitfelds declared with an integer type are unsigned
–no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
–no_exceptions	Disables C++ support for exception handling
–no_slash_comment	C++ style // comments are not accepted andgenerate errors
–prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
–incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DGHS	Predefine GHS as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.2.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed

Assembler Option	Description
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

3.1.2.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWARF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for themodules in this library is provided in the src/libstartup directory
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-nostartfiles	Controls the start files to be linked into the executable

3.1.3 DIAB Compiler/Assembler/Linker Options

3.1.3.1 DIAB Compiler Options

Compiler Option	Description
-tARMCORTEXM7MG:simple	Selects target processor (hardware single-precision, software double-precision floating-point)
-mthumb	Selects generating code that executes in Thumb state
-std=c99	Follows the C99 standard for C
-Oz	Like -O2 with further optimizations to reduce code size
-g	Generates DWARF 4.0 debug information
-fstandalone-debug	Emits full debug info for all types used by the program

Compiler Option	Description
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wsign-compare	Produce warnings when comparing signed type with unsigned type
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-Wunknown-pragmas	Issues a warning for unknown pragmas
-Wundef	Warns if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wextra	Enables some extra warning flags that are not enabled by '-Wall'
-Wall	Enables all of the most useful warnings (for historical reasons this option does not literally enable all warnings)
-pedantic	Emits a warning whenever the standard specified by the -std option requires a diagnostic
-Werror=implicit-function-declaration	Generates an error whenever a function is used before being declared
-fno-common	Compile common globals like normal definitions
-fno-signed-char	Char is unsigned
-fno-trigraphs	Do not process trigraph sequences
-V	Displays the current version number of the tool suite
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DDIAB	Predefine DIAB as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initalization in source file system.↩ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initalization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initalization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.3.2 DIAB Assembler Options

Assembler Option	Description
-mthumb	Selects generating code that executes in Thumb state
-Xpreprocess-assembly	Invokes C preprocessor on assembly files before running the assembler
-Xassembly-listing	Produces an .lst assembly listing file
-c	Stop after assembly and produce an object file for each source file

3.1.3.3 DIAB Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
linker_script_file.dld	Use linker_script_file.dld as the linker script. This script replaces the default linker script (rather than adding to it)
-m30	m2 + m4 + m8 + m16
-Xstack-usage	Gathers and display stack usage at link time
-Xpreprocess-lecl	Perform pre-processing on linker scripts
-Llibrary_path	Points to the libraries location for ARMV7EMMG to be used for linking
-lc	Links with the standard C library
-lm	Links with the math library

3.2 Files required for compilation

This section describes the include files required to compile, assemble and link the Serdes Driver for S32 microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR↩_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

Serdes Driver Files:

- ./Serdes_TS_T40D11M40I2R0/include/Serdes.h
- ./Serdes_TS_T40D11M40I2R0/include/Serdes_Ip.h
- ./Serdes_TS_T40D11M40I2R0/include/Serdes_Ip_HwAccess.h
- ./Serdes_TS_T40D11M40I2R0/include/Serdes_Ip_TrustedFunctions.h
- ./Serdes_TS_T40D11M40I2R0/include/Serdes_Ip_Types.h
- ./Serdes_TS_T40D11M40I2R0/include/Serdes_Ipw.h
- ./Serdes_TS_T40D11M40I2R0/include/Serdes_Ipw_Types.h
- ./Serdes_TS_T40D11M40I2R0/include/Serdes_Types.h
- ./Serdes_TS_T40D11M40I2R0/src/Serdes.c

- ./Serdes_TS_T40D11M40I2R0/src/Serdes_Ip.c
- ./Serdes_TS_T40D11M40I2R0/src/Serdes_Ip_HwAccess.c
- ./Serdes_TS_T40D11M40I2R0/src/Serdes_Ipw.c

Serdes Driver Generated Files (must be generated by the user using a configuration tool):

- Serdes_Ip_Cfg.h
- Serdes_Ip_PBCfg.h
- Serdes_Cfg.h
- Serdes_Ipw_PBCfg.h
- Serdes_Ip_CfgDefines.h
- Serdes_Ip_PBCfg.c
- Serdes_Cfg.c
- Serdes_Ipw_PBCfg.c

BaseNXP Files:

- BaseNXP_TS_T40D11M40I2R0\include\Mcal.h
- BaseNXP_TS_T40D11M40I2R0\include\Platform_Types.h
- BaseNXP_TS_T40D11M40I2R0\include\Soc_Ips.h
- BaseNXP_TS_T40D11M40I2R0\include\Std_Types.h
- BaseNXP_TS_T40D11M40I2R0\include\OsIf.h
- BaseNXP_TS_T40D11M40I2R0\generate_PC\include\modules.h

DET Files:

- Det_TS_T40D11M40I2R0\include\Det.h
- Det_TS_T40D11M40I2R0\src\Det.c

3.3 Setting up the plugins

The Serdes Driver was designed to be configured by using the EB Tresos Studio (version 27.1.0 b200625-0900 or later)

3.3.0.0.1 Location of various files inside the Serdes module folder:

- VSMD (Vendor Specific Module Definition) file in EB Tresos Studio XDM format:
  - Serdes_TS_T40D11M40I2R0\config\Serdes.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
  - Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g233a_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g234m_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g254a_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g274a_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g338m_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g339m_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g358a_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g359a_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g378a_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g379a_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g398a_bga525.epd
    Serdes_TS_T40D11M40I2R0\autosar\Serdes_s32g399a_bga525.epd
- Code Generation Templates for parameters without variation points:
  - Serdes_TS_T40D11M40I2R0\generate_PC\src\Serdes_Cfg.c
  - Serdes_TS_T40D11M40I2R0\generate_PC\src\Serdes_Ip_PBCfg.c
  - Serdes_TS_T40D11M40I2R0\generate_PC\src\Serdes_Ipw_PBCfg.c
  - Serdes_TS_T40D11M40I2R0\generate_PC\include\Serdes_Cfg.h
  - Serdes_TS_T40D11M40I2R0\generate_PC\include\Serdes_Ip_Cfg.h
  - Serdes_TS_T40D11M40I2R0\generate_PC\include\Serdes_Ip_CfgDefines.h
  - Serdes_TS_T40D11M40I2R0\generate_PC\include\Serdes_Ip_PBCfg.h
  - Serdes_TS_T40D11M40I2R0\generate_PC\include\Serdes_Ipw_PBCfg.h

3.3.0.0.2 Steps to generate the configuration:

1. Copy the following module folders into the Tresos plugins folder:
   - BaseNXP_TS_T40D11M40I2R0
   - Det_TS_T40D11M40I2R0
   - Mcu_TS_T40D11M40I2R0
   - Platform_TS_T40D11M40I2R0
   - Resource_TS_T40D11M40I2R0
   - Rte_TS_T40D11M40I2R0
   - Serdes_TS_T40D11M40I2R0
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB Tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files

Chapter 4

Function calls to module

- Function Calls during Start-up
- Function Calls during Shutdown
- Function Calls during Wake-up

4.1 Function Calls during Start-up

Serdes shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is Serdes_↩ Init(). The MCU module should be initialized before the Serdes is initialized. The API to be called for this purpose is Serdes_Init().

Serdes initalization is asynchronous, to complete the initializaton Serdes_MainFunction should be called multiple times until initialization is completed.

If PCIe mode is used, the PCIe controller must be initialized after the initialization of Serdes. The API to be called for this purpose is Pcie_Init().

4.2 Function Calls during Shutdown

N/A

4.3 Function Calls during Wake-up

N/A

Chapter 5

Module requirements

5.1 Exclusive areas to be defined in BSW scheduler

N/A

5.2 Exclusive areas not available on this platform

N/A

5.3 Peripheral Hardware Requirements

None.

5.4 ISR to configure within AutosarOS - dependencies

N/A

5.5 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

5.5.1 Without an Operating System

The macro `_USING_OS_AUTOSAROS_` must not be defined.

5.5.1.1 Using Software Vector Mode

The macro `_USE_SW_VECTOR_MODE_` must be defined and the ISR macro is defined as:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

5.5.1.2 Using Hardware Vector Mode

The macro `_USE_SW_VECTOR_MODE_` must not defined and the ISR macro is defined as:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, the drivers' interrupt handlers must also handle the context save and restore.

5.5.2 With an Operating System

Please refer to your OS documentation for description of the ISR macro.

5.6 Other AUTOSAR modules - dependencies

- **MCU**: The MCU driver provides services for basic microcontroller initialization, power down functionality, reset and microcontroller specific functions required by other MCAL software modules. The clocks need to be initialized prior to using the Serdes driver.
- **DET**: The DET module is used for enabling Development error detection. The API function used is Det↩ _ReportError(). The activation / deactivation of Development error detection is configurable using the '↩ Serdes_DevErrorDetect' configuration parameter.
- **BaseNXP**: The BASENXP module contains the common files/definitions needed by all MCAL modules.
- **PLATFORM**: The PLATFORM module is required for interrupt service routines.
- **RESOURCE** :Resource module is used to select microcontroller's derivatives.
- **RTE** :The RTE module is needed for implementing data consistency of exclusive areas that are used by Serdes module.

5.7 Data Cache Restrictions

N/A

5.8 User Mode support

- User Mode configuration in the module
- User Mode configuration in AutosarOS

5.8.1 User Mode configuration in the module

The Serdes module can run in user mode if SerdesEnableUserModeSupport from the configuration is enabled.

#	Function syntax	Description	Available via
1	boolean Serdes_Ip_SsModeConfigure_TrustedCall(uint8 instance, Serdes_Ip_ModeType mode, uint32 intclks)	Configures serdes mode of operation	Serdes_Ip_TrustedFunctions.h

5.8.2 User Mode configuration in AutosarOS

When User mode is enabled, the driver may has the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header `<IpName>_Ip_TrustedFunctions.h`. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter User Mode configuration in the module for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

`Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)`

That is the result of macro expansion `OsIf_Trusted_Call` in driver code:

`#define` OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)

So, the following steps need to be done in AutosarOS:

- Ensure `MCAL_ENABLE_USER_MODE_SUPPORT` macro is defined in the build system or somewhere global.
- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to visible in `Os.h` for the driver to call them. They will do the marshalling of the parameters and call `CallTrustedFunction()` in OS specific manner.
- `CallTrustedFunction()` will switch to privileged mode and call `TRUSTED_<Function_Name>()`.
- `TRUSTED_<Function_Name>()` function is also defined and declared in Integration/User code. It will unmarshalling of the parameters to call `<Function_Name>`() of driver. The `<Function_Name>`() functions are already defined in driver and declared in `<IpName>_Ip_TrustedFunctions.h`. This header should be included in OS for OS call and indexing these functions.

See the sequence chart below for an example calling `Linflexd_Uart_Ip_Init_Privileged()` as a trusted function.

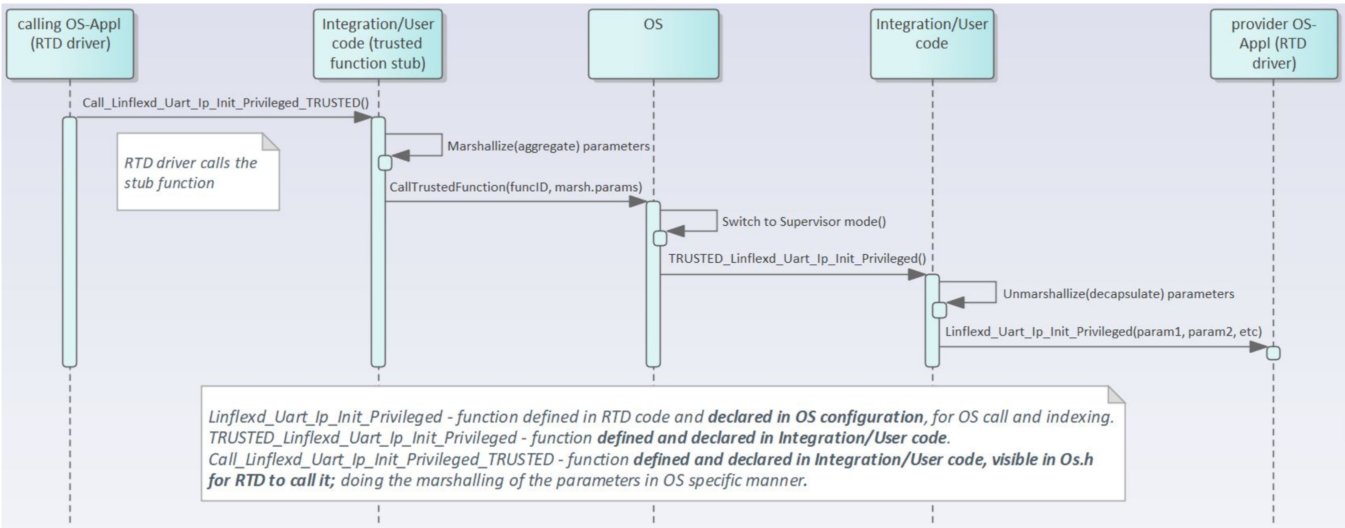

Figure 5.1 Example sequence chart for calling `Linflexd_Uart_Ip_Init_Privileged` as trusted function

5.9 Multicore support

The **Serdes** driver implements the **Autosar 4.4 MCAL Multicore Distribution** according to type II, in which the mappable element is set to Serdes Driver Object. For additional details, please refer to **AUTOSAR_EXP_BSWDistributionGuide**.

The **Serdes** driver and the mappable elements can be allocated to zero, one or several ECUC partitions, by means of **SerdesEcucPartionRef**. If the **Serdes** is mapped to zero ECUC partitions, the **Serdes** behavior reverts to single-core implementation, similar to previous Autosar versions. If the **Serdes** is mapped to one or more ECUC partitions, the **Serdes** enforces the following multi-core assumptions:

1. The **Serdes** driver assumes there is a single EcucPartition allocated per core. Internally, the module will use the Core ID returned by GetCoreID API to reference the appropriate global data and configuration elements.

2. The **Serdes** driver assumes the EcucCoreIDs are defined in a compact/consecutive order, starting from zero. The rationale is that the number of EcucPartitions is used for dimensioning the **Serdes** internal variables and the EcucCoreIDs are used for indexing those variables. (AR-86601 Zero based and dense IDs for OS-Cores and OSApplications)

3. The **Serdes** driver assumes that initialization is performed on each core, Serdes_Init() is called separately for each core.

4. The **Serdes** driver will check upon each API call if the requested resource is configured to be available on the current core, if DET error reporting is enabled.

5. The **Serdes** driver requires that all variables in NonCacheable MemMap sections be allocated accordingly, to avoid data corruption in multicore context.

6. The **Serdes** driver assumes that RTE module implements the EXCLUSIVE AREAS to be core-aware only. The rationale is that the module implementation ensures data integrity by separating the mappable elements for different cores already, thus implementing the EXCLUSIVE AREAS in a blocking manner (ex: spin-lock) on a multicore scope, might affect the performance of the drivers on the two cores, although they might access separate HW elements. For single-core scope, the EXCLUSIVE AREAS keep the same purpose as on previous AUTOSAR implementations. (to be updated per **Serdes** usecase, to be detailed/removed if some modules require such kind of functionality for critical features which cannot be atomically shared among cores).

Chapter 6

Main API Requirements

6.1 Main function calls within BSW scheduler

The Serdes Driver supports a main function that can be configured to be scheduled by the BSW Scheduler:

void Serdes_MainFunction(void)

Call rate depends on target application and can be configured from SerdesGeneral/SerdesMainFunctionPeriod.

6.2 API Requirements

Serdes_Init() function should be called before calling Serdes_MainFunction(). Serdes_Init() will begin the initialization and Serdes_MainFunction() checks for state changes to advance the initialization process until it is complete.

6.3 Calls to Notification Functions, Callbacks, Callouts

Serdes module provides user-configurable notifications:

- SerdesJobDone
- SerdesJobFailed

SerdesJobDone is used by the underlying Serdes module to report the successful end of serdes initialization. SerdesJobFailed is used by the underlying Serdes module to report the failure of serdes initialization.

Both notifications are optional and are user-configurable.

Chapter 7

Memory allocation

- Sections to be defined in Serdes_MemMap.h
- Linker command file

7.1 Sections to be defined in Serdes_MemMap.h

Section name	Type of section	Description
SERDES_START_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start of Memory Section for Rule constant with attributes that reside in module configuration that are not Boolean, 8 bits, 16bits or 32 bits
SERDES_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	End of Memory Section for Config Data
SERDES_START_SEC_CODE	Code	Start of memory Section for Code
SERDES_STOP_SEC_CODE	Code	End of memory Section for Code
SERDES_START_SEC_CONST_UNSPECIFIED	Const	Start of memory Section for constants that are not boolean, 8 bits, 16 bits or 32 bits
SERDES_STOP_SEC_CONST_UNSPECIFIED	Const	End of memory Section for const
SERDES_START_SEC_VAR_INIT_8	Variables	Start of Memory Section for 8 bits variables that are initialized
SERDES_STOP_SEC_VAR_INIT_8	Variables	End of Memory Section for var
SERDES_START_SEC_VAR_CLEARED_8	Variables	Start of Memory Section for 8 bits variables that are not initialized
SERDES_STOP_SEC_VAR_CLEARED_8	Variables	End of above section.
SERDES_START_SEC_VAR_CLEARED_UNSPECIFIED	Variables	Start of Memory Section for variables that are not boolean, 8 bits, 16 bits or 32 bits
SERDES_STOP_SEC_VAR_CLEARED_UNSPECIFIED	Variables	End of above section.

7.2 Linker command file

Memory shall be allocated for every section defined in the driver's "<Module>"_MemMap.h.

Chapter 8

Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section Files Required for Compilation
2. Allocate the proper memory sections in the driver's memory map header file ("<Module>"_MemMap.h) and linker command file. For more details refer to section Sections to be defined in <Module>_MemMap.h
3. Compile & build the module with all the dependent modules. For more details refer to section Building the Driver

Chapter 9

External assumptions for driver

The section presents requirements that must be complied with when integrating the SERDES driver into the application.

External Assumption Req ID	External Assumption Text
EA_RTD_00071	If interrupts are locked, a centralized function pair to lock and unlock interrupts shall be used.
EA_RTD_00082	When caches are enabled and data buffers are allocated in cacheable memory regions the buffers involved in DMA transfer shall be aligned with both start and end to cache line size. Note: **Rationale**: This ensures that no other buffers/variables compete for the same cache lines.
EA_RTD_00092	The integrator shall allocate a single EcucPartition per core or the partition in which the Serdes is allocated shall be exclusively mapped to a core. Note: Internally, the Serdes will use the Core ID returned by GetCoreID API to reference the appropriate global data and configuration elements, that is why a core should reference only one configured partition.
EA_RTD_00093	The application shall define EcucCoreIDs in a compact/consecutive order, starting from zero.
EA_RTD_00094	When multicore support is enabled, the application shall call Serdes_Init() for each core, using the dedicated configuration pointer for that core.
EA_RTD_00096	The application shall pass the correct initialization pointer, specific to the partition in which the driver is to be used.
EA_RTD_00113	When RTD drivers are integrated with AutosarOS and User mode support is enabled, the integrator shall assure that the definition and declaration of all RTD functions needed to be called as trusted functions follow the naming convention Call<Function_Name>TRUSTED(parameter1,parameter2,...) in Integration/User code. They need to visible in Os.h for the driver to call them. They will call RTD <Function_Name>() as trusted functions in OS specific manner.

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support